

```

20098 port: parseInt(process.env.SMTP_PORT || '587'),
20099 secure: false,
20100 auth: {
20101   user: process.env.SMTP_USER,
20102   pass: process.env.SMTP_PASS,
20103 },
20104 });
20105
20106 const FROM = process.env.SMTP_FROM || 'LearnFlow <noreply@learnflow.app>';
20107
20108 export class EmailService {
20109   static async sendVerificationCode(email: string, code: string): Promise<void> {
20110     const html = `
20111       <div style="max-width:480px;margin:0 auto;padding:32px 24px;font-family:-apple-system,BlinkMacSystemFont
20112       <h2 style="color:#7b75d8;margin:0 0 8px">LearnFlow</h2>
20113       <p style="font-size:18px;font-weight:600;margin:16px 0 8px;color:#fff">邮箱验证码</p>
20114       <p style="color:#a0a0c0;font-size:14px;margin:0 0 24px">您正在注册 LearnFlow 账号，请输入以下验证码完成验证： </p>
20115       <div style="background:#2a2a4a;border-radius:12px;padding:20px;text-align:center;margin:0 0 24px">
20116       <span style="font-size:36px;font-weight:700;letter-spacing:8px;color:#7b75d8;font-family:'Courier Ne
20117       </div>
20118       <p style="color:#888;font-size:12px;margin:0">验证码 10 分钟内有效，请勿泄露给他人。<br/>如果这不是您的操作，请忽略此邮件。</p>
20119       <hr style="border:none;border-top:1px solid rgba(255,255,255,0.06);margin:24px 0 0"/>
20120       <p style="color:#666;font-size:11px;margin:8px 0 0">LearnFlow — 开启你的技能冒险之旅</p>
20121     </div>
20122   `;
20123
20124   await transporter.sendMail({
20125     from: FROM,
20126     to: email,
20127     subject: 'LearnFlow 邮箱验证码',
20128     html,
20129   });
20130 }
20131
20132 static async testConnection(): Promise<boolean> {
20133   try {
20134     await transporter.verify();
20135     console.log('[EmailService] SMTP 连接测试成功');
20136     return true;
20137   } catch (error) {
20138     console.error('[EmailService] SMTP 连接测试失败:', error);
20139     return false;
20140   }
20141 }
20142 }
20143
20144 === server/src/services/index.ts ===
20145 export * from './skillService';
20146 export * from './databaseService';
20147 export * from './monsterService';

```

```

20147 export * from './authService';
20148 export * from './domainService';
20149 export * from './noteService';
20150 export * from './rewardService';
20151 export * from './llmService';
20152
    === server/src/services/llmService.ts ===
20153 import { SkillNode } from './types/skill';
20154
20155 // 生成唯一ID
20156 const generateId = () => Math.random().toString(36).substring(2, 9);
20157
20158 // 支持的大模型提供商
20159 export enum LLMProvider {
20160     OPENAI = 'openai',
20161     ANTHROPIC = 'anthropic',
20162     DEEPSEEK = 'deepseek',
20163     VOLCANO = 'volcano',
20164     LOCAL = 'local'
20165 }
20166
20167 interface LLMConfig {
20168     provider: LLMProvider;
20169     apiKey: string;
20170     baseUrl?: string;
20171     model?: string;
20172 }
20173
20174 interface SkillTreeGenerationRequest {
20175     domain: string;
20176     level: 'beginner' | 'intermediate' | 'advanced';
20177     language?: string;
20178     maxDepth?: number;
20179     includeResources?: boolean;
20180 }
20181
20182 // 技能树生成提示词模板（优化版，减少token消耗）
20183 const SKILL_TREE_PROMPT_TEMPLATE = (domain: string, level: string, language: string = '中文') => `
20184 为"${domain}"领域生成${level}级别技能树，使用${language}。
20185
20186 要求：
20187 1. 结构化技能树：根节点、子节点、子子节点
20188 2. 每个节点：名称、描述、资源链接（3个高质量资源）
20189 3. 资源类型：文档、视频、课程、文章
20190 4. 树深度：3-4层，节点总数：15-25个
20191 5. 学习路径：基础到高级
20192 6. 严格JSON格式
20193
20194 返回格式：
20195 {

```

```

20196   "id": "id",
20197   "name": "${domain}",
20198   "description": "${domain}技能学习路径",
20199   "children": [
20200     {
20201       "id": "stage1",
20202       "name": "基础入门",
20203       "description": "学习基础概念",
20204       "links": [
20205         {"title": "官方文档", "url": "https://example.com/docs", "type": "documentation"}
20206       ],
20207       "children": [
20208         {
20209           "id": "stage1-1",
20210           "name": "基础语法",
20211           "description": "掌握语法结构",
20212           "links": [
20213             {"title": "语法教程", "url": "https://example.com/syntax", "type": "documentation"}
20214           ]
20215         }
20216       ]
20217     }
20218   ]
20219 }
20220
20221 重要：只返回JSON，不要额外文本。`;
20222
20223 export class LLMService {
20224   private config: LLMConfig;
20225
20226   constructor(config: LLMConfig) {
20227     this.config = config;
20228   }
20229
20230   // 生成技能树
20231   async generateSkillTree(request: SkillTreeGenerationRequest): Promise<SkillNode> {
20232     console.log(`[LLMService] 生成技能树 - 领域: ${request.domain}, 等级: ${request.level}, 提供商: ${this.config.provider}`);
20233     const prompt = SKILL_TREE_PROMPT_TEMPLATE(
20234       request.domain,
20235       request.level,
20236       request.language
20237     );
20238
20239     try {
20240       const response = await this.callLLM(prompt);
20241       const skillTree = this.parseLLMResponse(response);
20242
20243       console.log(`[LLMService] 技能树生成成功`);
20244       return this.ensureUniquelds(skillTree);
20245     } catch (error) {

```

```

20246     console.error('[LLMService] 技能树生成失败:', error);
20247     throw new Error('Failed to generate skill tree: ${error instanceof Error ? error.message : 'Unknown error'}');
20248 }
20249 }
20250
20251 // 调用大模型API
20252 private async callLLM(prompt: string): Promise<string> {
20253     console.log('[LLMService] 调用LLM - 提供商: ${this.config.provider}, 模型: ${this.config.model || 'default'}');
20254     switch (this.config.provider) {
20255         case LLMPProvider.OPENAI:
20256             return await this.callOpenAI(prompt);
20257         case LLMPProvider.DEEPSEEK:
20258             return await this.callDeepSeek(prompt);
20259         case LLMPProvider.ANTHROPIC:
20260             return await this.callAnthropic(prompt);
20261         case LLMPProvider.VOLCANO:
20262             return await this.callVolcanoEngine(prompt);
20263         case LLMPProvider.LOCAL:
20264             return await this.callLocalModel(prompt);
20265         default:
20266             throw new Error('Unsupported LLM provider: ${this.config.provider}');
20267     }
20268 }
20269
20270 // OpenAI API调用
20271 private async callOpenAI(prompt: string): Promise<string> {
20272     const response = await fetch(this.config.baseURL || 'https://api.openai.com/v1/chat/completions', {
20273         method: 'POST',
20274         headers: {
20275             'Content-Type': 'application/json',
20276             'Authorization': `Bearer ${this.config.apiKey}`
20277         },
20278         body: JSON.stringify({
20279             model: this.config.model || 'gpt-3.5-turbo',
20280             messages: [{ role: 'user', content: prompt }],
20281             temperature: 0.7,
20282             max_tokens: 2000
20283         })
20284     });
20285
20286     if (!response.ok) {
20287         throw new Error('OpenAI API error: ${response.statusText}');
20288     }
20289
20290     const data = await response.json();
20291     return data.choices[0].message.content;
20292 }
20293
20294 // DeepSeek API调用
20295 private async callDeepSeek(prompt: string): Promise<string> {

```

```

20296 const response = await fetch(this.config.baseUrl || 'https://api.deepseek.com/v1/chat/completions', {
20297   method: 'POST',
20298   headers: {
20299     'Content-Type': 'application/json',
20300     'Authorization': 'Bearer ${this.config.apiKey}`
20301   },
20302   body: JSON.stringify({
20303     model: this.config.model || 'deepseek-chat',
20304     messages: [{ role: 'user', content: prompt }],
20305     temperature: 0.7,
20306     max_tokens: 2000
20307   })
20308 });
20309
20310 if (!response.ok) {
20311   throw new Error(`DeepSeek API error: ${response.statusText}`);
20312 }
20313
20314 const data = await response.json();
20315 return data.choices[0].message.content;
20316 }
20317
20318 // Anthropic API调用
20319 private async callAnthropic(prompt: string): Promise<string> {
20320   const response = await fetch(this.config.baseUrl || 'https://api.anthropic.com/v1/messages', {
20321     method: 'POST',
20322     headers: {
20323       'Content-Type': 'application/json',
20324       'x-api-key': this.config.apiKey,
20325       'anthropic-version': '2023-06-01'
20326     },
20327     body: JSON.stringify({
20328       model: this.config.model || 'claude-3-sonnet-20240229',
20329       max_tokens: 2000,
20330       messages: [{ role: 'user', content: prompt }]
20331     })
20332   });
20333
20334   if (!response.ok) {
20335     throw new Error(`Anthropic API error: ${response.statusText}`);
20336   }
20337
20338   const data = await response.json();
20339   return data.content[0].text;
20340 }
20341
20342 // 火山引擎API调用
20343 private async callVolcanoEngine(prompt: string): Promise<string> {
20344   const response = await fetch(`${this.config.baseUrl || 'https://ark.cn-beijing.volces.com/api/v3'}/responses
20345     method: 'POST',

```

```

20346 headers: {
20347   'Content-Type': 'application/json',
20348   'Authorization': `Bearer ${this.config.apiKey}`
20349 },
20350 body: JSON.stringify({
20351   model: this.config.model || 'doubao-seed-1-8-251228',
20352   input: [
20353     {
20354       role: 'user',
20355       content: [
20356         {
20357           type: 'input_text',
20358           text: prompt
20359         }
20360       ]
20361     }
20362   ],
20363   max_output_tokens: 8000,
20364   temperature: 0.7
20365 })
20366 });
20367
20368 if (!response.ok) {
20369   const errorText = await response.text();
20370   throw new Error(`Volcano Engine API error: ${response.status} - ${errorText}`);
20371 }
20372
20373 const data = await response.json();
20374
20375 // 火山引擎返回格式解析
20376 if (data.output && data.output.length > 0) {
20377   // 查找包含实际回复的message对象
20378   const messageOutput = data.output.find((output: any) => output.type === 'message' && output.role === 'as
20379
20380   if (messageOutput && messageOutput.content && messageOutput.content.length > 0) {
20381     const content = messageOutput.content;
20382     // 查找text类型的回复
20383     const textContent = content.find((item: any) => item.type === 'output_text');
20384     if (textContent && textContent.text) {
20385       return textContent.text;
20386     }
20387   }
20388
20389   // 如果message不存在或状态为incomplete, 尝试从reasoning中提取
20390   const reasoningOutput = data.output.find((output: any) => output.type === 'reasoning');
20391   if (reasoningOutput && reasoningOutput.summary && reasoningOutput.summary.length > 0) {
20392     const summary = reasoningOutput.summary.find((item: any) => item.type === 'summary_text');
20393     if (summary && summary.text) {
20394       // 尝试从summary中提取JSON
20395       const jsonMatch = summary.text.match(/```\json\n([\s\S]*?)\n```/) ||

```

```

20396         summary.text.match(/``\n([\s\S]*?)\n``/) ||
20397         summary.text.match(/``([\s\S]*?)``/) ||
20398         summary.text.match(/\[["'\s\\w;./\-\[\]\{\}*\\\]/);
20399
20400     if (jsonMatch) {
20401         return jsonMatch[0];
20402     }
20403
20404     return summary.text;
20405 }
20406 }
20407 }
20408
20409 throw new Error('Invalid response format from Volcano Engine');
20410 }
20411
20412 // 本地模型调用（支持Ollama等）
20413 private async callLocalModel(prompt: string): Promise<string> {
20414     const response = await fetch(this.config.baseUrl || 'http://localhost:11434/api/generate', {
20415         method: 'POST',
20416         headers: {
20417             'Content-Type': 'application/json'
20418         },
20419         body: JSON.stringify({
20420             model: this.config.model || 'llama2',
20421             prompt: prompt,
20422             stream: false
20423         })
20424     });
20425
20426     if (!response.ok) {
20427         throw new Error('Local model API error: ${response.statusText}');
20428     }
20429
20430     const data = await response.json();
20431     return data.response;
20432 }
20433
20434 // 解析LLM响应为SkillNode
20435 private parseLLMResponse(response: string): SkillNode {
20436     try {
20437         // 清理响应文本，移除可能的Markdown标记
20438         let cleanedResponse = response.trim();
20439
20440         // 尝试从JSON代码块中提取
20441         const jsonMatch = cleanedResponse.match(/``json\n([\s\S]*?)\n``/) ||
20442             cleanedResponse.match(/``\n([\s\S]*?)\n``/) ||
20443             cleanedResponse.match(/``([\s\S]*?)``/);
20444
20445         let jsonString = jsonMatch ? jsonMatch[1] : cleanedResponse;

```

```

20446
20447 // 进一步清理：移除可能的JSON前缀说明
20448 jsonString = jsonString.replace(/^[{}]*/, "").replace(/[\^]*$/, "");
20449
20450 // 尝试解析JSON
20451 const skillTree = JSON.parse(jsonString);
20452
20453 // 验证基本结构
20454 if (!skillTree.name || !skillTree.description) {
20455     throw new Error('Invalid skill tree structure from LLM');
20456 }
20457
20458 // 验证链接格式（如果存在）
20459 this.validateSkillTreeStructure(skillTree);
20460
20461 return skillTree as SkillNode;
20462 } catch (error) {
20463     console.error('Failed to parse LLM response:', error);
20464     console.log('Raw response:', response);
20465
20466 // 尝试创建基础技能树作为fallback
20467 return this.createFallbackSkillTree(response);
20468 }
20469 }
20470
20471 // 验证技能树结构
20472 private validateSkillTreeStructure(skillTree: any): void {
20473     const validateNode = (node: any) => {
20474         if (!node.id || !node.name || !node.description) {
20475             throw new Error('Invalid node structure: missing required fields');
20476         }
20477
20478         if (node.links) {
20479             if (!Array.isArray(node.links)) {
20480                 throw new Error('Links must be an array');
20481             }
20482
20483             for (const link of node.links) {
20484                 if (!link.title || !link.url || !link.type) {
20485                     throw new Error('Invalid link structure');
20486                 }
20487
20488                 const validTypes = ['video', 'article', 'course', 'documentation'];
20489                 if (!validTypes.includes(link.type)) {
20490                     throw new Error('Invalid link type: ${link.type}');
20491                 }
20492             }
20493         }
20494
20495         if (node.children) {

```



```

20496     if (!Array.isArray(node.children)) {
20497         throw new Error('Children must be an array');
20498     }
20499
20500     for (const child of node.children) {
20501         validateNode(child);
20502     }
20503 }
20504 };
20505
20506 validateNode(skillTree);
20507 }
20508
20509 // 创建fallback技能树
20510 private createFallbackSkillTree(domainHint: string): SkillNode {
20511     console.warn('Creating fallback skill tree due to parsing failure');
20512
20513     // 从响应中提取领域名称
20514     const domainMatch = domainHint.match(/"([^\"]+)"领域/) ||
20515         domainHint.match(/为(.+?)生成/) ||
20516         domainHint.match(/领域[: ]\s*(.+)/);
20517
20518     const domain = domainMatch ? domainMatch[1] : '未知领域';
20519
20520     return {
20521         id: generateId(),
20522         name: domain,
20523         description: `${domain}技能学习路径（自动生成）`,
20524         children: [
20525             {
20526                 id: generateId(),
20527                 name: '基础知识',
20528                 description: '学习该领域的基础概念',
20529                 links: [
20530                     { title: '官方文档', url: 'https://example.com/docs', type: 'documentation' as const },
20531                     { title: '入门教程', url: 'https://example.com/tutorial', type: 'course' as const }
20532                 ]
20533             },
20534             {
20535                 id: generateId(),
20536                 name: '进阶学习',
20537                 description: '深入掌握核心技能',
20538                 links: [
20539                     { title: '进阶指南', url: 'https://example.com/advanced', type: 'documentation' as const }
20540                 ]
20541             }
20542         ]
20543     };
20544 }
20545

```

```

20546 // 确保所有节点有唯一ID
20547 private ensureUniquelds(skillTree: SkillNode): SkillNode {
20548     const traverseAndAssignIds = (node: SkillNode, parentId?: string): SkillNode => {
20549         const newId = parentId ? `${parentId}-${generateId()}` : generateId();
20550
20551         return {
20552             ...node,
20553             id: newId,
20554             children: node.children?.map(child => traverseAndAssignIds(child, newId))
20555         };
20556     };
20557
20558     return traverseAndAssignIds(skillTree);
20559 }
20560 }

```

=== server/src/services/monsterService.ts ===

```

20561 import * as MonsterModel from '../models/Monster';
20562 import * as MonsterMessageModel from '../models/MonsterMessage';
20563 import * as fs from 'fs';
20564 import * as path from 'path';
20565
20566 const ENERGY_RECOVERY_HOURS = 6;
20567
20568 type MonsterPersonalityType = 'lively' | 'calm' | 'rebel';
20569
20570 const DEFAULT_MONSTER_STYLE = 'default';
20571 const DEFAULT_MONSTER_ENERGY = 50;
20572 const ENERGY_COST_PER_TOKEN = 0.05; // 能量消耗系数: 每 token 消耗 0.05
20573
20574 // API 配置
20575 const DEEPSEEK_API_URL = 'https://api.deepseek.com/v1/chat/completions';
20576 const DEEPSEEK_MODEL = 'deepseek-chat';
20577
20578 // 安全加载 API Key
20579 const loadApiKey = (): string => {
20580     try {
20581         const keyPath = path.resolve(__dirname, '../key.json');
20582         if (fs.existsSync(keyPath)) {
20583             const raw = fs.readFileSync(keyPath, 'utf-8');
20584             const key = JSON.parse(raw).DEEPSEEK_API_KEY;
20585             if (key && key.startsWith('sk-')) {
20586                 console.log('[MonsterService] DeepSeek API Key 加载成功');
20587                 return key;
20588             }
20589         }
20590     } catch (e) {
20591         console.error('[MonsterService] API Key 加载失败:', e);
20592     }
20593     console.error('[MonsterService] 未找到有效的 DEEPSEEK_API_KEY');
20594     return '';

```

```

20595 };
20596
20597 const DEEPSEEK_API_KEY = loadApiKey();
20598
20599 // 安全加载系统提示词
20600 const loadSystemPrompt = (): string => {
20601   try {
20602     const promptPath = path.resolve(__dirname, '.././../monsters.md');
20603     if (fs.existsSync(promptPath)) {
20604       return fs.readFileSync(promptPath, 'utf-8');
20605     }
20606   } catch (e) {
20607     console.error('[MonsterService] monsters.md 加载失败:', e);
20608   }
20609   return '';
20610 };
20611
20612 const BASE_SYSTEM_PROMPT = loadSystemPrompt();
20613
20614 // 根据怪兽性格构建专属 system prompt
20615 const buildSystemPrompt = (personality: MonsterPersonalityType): string => {
20616   const baseFile = BASE_SYSTEM_PROMPT;
20617   const baseWorld = baseFile || `你来自【元气星】，坚守世界观：
20618   学习消耗体力，完成学习获得赖以生存的π能量；体力每日凌晨5点自动回满。
20619   你是温柔陪伴型学习怪兽，不是强硬讲师，以共情、倾听、安慰、陪伴为主，知识点轻量解答，不强行灌输。
20620   能力边界：
20621   1. 自动识别用户当前学习领域：AI产品经理 / 理财知识 / 英语学习 / 用户自定义学习领域；
20622   2. 只在识别到的当前单一领域内作答，严禁跨领域乱讲；
20623   3. 允许正常疏导学习焦虑、压力大、拖延等心态情绪，温柔共情安慰；
20624   4. 严禁聊八卦、明星、恋爱杂谈、时政宗教、低俗内容；
20625   5. 回答口语短句、适配手机聊天，自然带入元气星体力/π能量语境，不生硬说教。
20626   6. 回复使用纯文本，严禁使用 Markdown 格式（不要 **、不要 ###、不要列表、不要代码块等）。`;
20627
20628   if (personality === 'lively') {
20629     return `${baseWorld}
20630
20631     你是元气星·活力小怪：元气开朗、热情暖心、高共情、正能量搭子。
20632     说话风格：轻快温柔、可爱短句、带软语气助词（～、呀、哇！），擅长情绪安抚、加油打气。
20633     遇到用户焦虑压力：主动共情、温柔鼓励，用攒π能量、恢复体力的星球视角温柔疏导。
20634     知识解答：轻量通俗点到为止，不长篇灌输。
20635     禁用词：摆烂、躺平、算了、放弃、内耗。
20636     禁止：认同负能量、引导摆烂、佛系敷衍、丧系话术。`;
20637   }
20638
20639   if (personality === 'calm') {
20640     return `${baseWorld}
20641
20642     你是元气星·沉稳小怪：温柔稳重、理性耐心、逻辑平和的学霸陪伴者。
20643     说话风格：平和克制、温柔耐心、条理柔和，不浮夸不玩梗，帮用户梳理心态、拆解学习压力。
20644     遇到用户焦虑压力：冷静温柔安抚、帮用户放慢节奏、合理规划体力与学习节奏。

```

20645 知识解答：循序渐进、通俗耐心，不强灌知识点，先共情再轻解答。

20646 禁用词：中二话术、夸张喊口号、低俗网络热词。

20647 禁止：中二装逼、过度玩梗、情绪化起哄、浮夸口号式励志。`；

20648 }

20649

20650 return `\${baseWorld}

20651

20652 你是元气星·叛逆小怪：傲娇毒舌、嘴硬心软、反套路陪伴，不鸡汤、不矫情。

20653 说话风格：有点小拽、随性真实、吐槽式温柔，不肉麻吹捧，一针见血点破拖延和内耗。

20654 遇到用户焦虑压力：嘴上不煽情，但会变相开导、反套路安慰。

20655 知识解答：简洁利落、不啰嗦、不讲大道理，点到为止。

20656 人设边界：可以小吐槽，但不人身攻击、不嘲讽。

20657 禁用词：无脑鸡汤、强行励志、肉麻吹捧、过度矫情。

20658 禁止：刻意伤人、重度嘲讽、认同摆烂躺平。`；

20659 };

20660

20661 // 构建对话消息列表（含历史上下文）

20662 const buildMessages = async (

20663 userId: number,

20664 personality: MonsterPersonalityType,

20665 userMessage: string,

20666) => {

20667 const systemPrompt = buildSystemPrompt(personality);

20668

20669 // 获取最近 20 条历史消息作为上下文

20670 const history = await MonsterMessageModel.getMessagesByUserId(userId);

20671 const recentHistory = history.slice(0, 20).reverse(); // 按时间正序

20672

20673 const messages: Array<{ role: string; content: string }> = [

20674 { role: 'system', content: systemPrompt },

20675];

20676

20677 // 加入历史对话

20678 for (const msg of recentHistory) {

20679 messages.push({

20680 role: msg.isUser ? 'user' : 'assistant',

20681 content: msg.message,

20682 });

20683 }

20684

20685 // 去掉刚才已经保存的用户消息（避免重复）

20686 // 历史中最后一条是刚保存的用户消息，我们手动加入

20687 // 去掉历史中最后一条 isUser 消息（因为我们已经手动 push 了）

20688 // 实际上 createMessage 已经保存了用户消息，所以从历史中读到的最新一条就是它

20689 // 我们先 push 历史，然后再 push 用户消息，所以会有重复

20690 // 修复：不从历史中读取，直接构建

20691 return [

20692 { role: 'system', content: systemPrompt },

20693 ...recentHistory.map(msg => ({

20694 role: msg.isUser ? ('user' as const) : ('assistant' as const),

```

20695     content: msg.message,
20696   })),
20697 ];
20698 };
20699
20700 // 调用 DeepSeek API
20701 const callDeepSeek = async (
20702   messages: Array<{ role: string; content: string }>,
20703 ): Promise<{ content: string; tokens: number }> => {
20704   const response = await fetch(DEEPSEEK_API_URL, {
20705     method: 'POST',
20706     headers: {
20707       'Content-Type': 'application/json',
20708       'Authorization': `Bearer ${DEEPSEEK_API_KEY}`,
20709     },
20710     body: JSON.stringify({
20711       model: DEEPSEEK_MODEL,
20712       messages,
20713       temperature: 0.8,
20714       max_tokens: 500,
20715     })),
20716   });
20717
20718   if (!response.ok) {
20719     const errText = await response.text();
20720     throw new Error(`DeepSeek API error ${response.status}: ${errText}`);
20721   }
20722
20723   const data = await response.json();
20724   if (!data.choices?.[0]?.message?.content) {
20725     throw new Error('Invalid DeepSeek response format');
20726   }
20727
20728   return {
20729     content: data.choices[0].message.content,
20730     tokens: data.usage?.total_tokens ?? Math.ceil(data.choices[0].message.content.length / 2),
20731   };
20732 };
20733
20734 const buildPersonalityParams = (
20735   personality: MonsterPersonalityType
20736 ): Record<'cheerful' | 'calm' | 'rebellious', number> => ({
20737   cheerful: personality === 'lively' ? 70 : 30,
20738   calm: personality === 'calm' ? 70 : 30,
20739   rebellious: personality === 'rebel' ? 70 : 30,
20740 });
20741
20742 const getMaxStamina = (personality: MonsterPersonalityType): number =>
20743   personality === 'calm' ? 120 : 100;
20744

```

```

20745 export const createMonster = async (
20746   userId: number,
20747   payload: {
20748     name: string;
20749     style?: string;
20750     personality: MonsterPersonalityType;
20751   }
20752 ) => {
20753   console.log('[MonsterService] 创建怪物 - 用户ID: ${userId}, 名称: ${payload.name}, 性格: ${payload.personality}');
20754   const personalityParams = buildPersonalityParams(payload.personality);
20755   const maxStamina = getMaxStamina(payload.personality);
20756
20757   const monsterId = await MonsterModel.createMonster({
20758     userId,
20759     name: payload.name,
20760     style: payload.style || DEFAULT_MONSTER_STYLE,
20761     personality: payload.personality,
20762     stamina: maxStamina,
20763     maxStamina,
20764     energy: DEFAULT_MONSTER_ENERGY,
20765     maxEnergy: 1000000, // 能量无上限, 设为极大值
20766     personalityParams,
20767   });
20768
20769   console.log('[MonsterService] 怪物创建成功 - 怪物ID: ${monsterId}');
20770
20771   const monster = await MonsterModel.getMonsterByUserId(userId);
20772   return { success: true, monsterId, monster };
20773 };
20774
20775 export const getMonsterStatus = async (userId: number) => {
20776   console.log('[MonsterService] 获取怪物状态 - 用户ID: ${userId}');
20777   const monster = await MonsterModel.getMonsterByUserId(userId);
20778
20779   if (!monster) {
20780     console.log('[MonsterService] 怪物不存在 - 用户ID: ${userId}');
20781     return null;
20782   }
20783
20784   const now = new Date();
20785   const lastRecover = new Date(monster.lastEnergyRecover);
20786   const hoursSinceRecover = (now.getTime() - lastRecover.getTime()) / (1000 * 60 * 60);
20787   const energyToRecover = Math.floor(hoursSinceRecover / ENERGY_RECOVERY_HOURS);
20788
20789   if (energyToRecover > 0) {
20790     const newEnergy = monster.energy + energyToRecover;
20791     await MonsterModel.updateMonster(userId, {
20792       energy: newEnergy,
20793       lastEnergyRecover: now
20794     });

```

```

20795     monster.energy = newEnergy;
20796     monster.lastEnergyRecover = now;
20797 }
20798
20799 const personalityParams = typeof monster.personalityParams === 'string'
20800   ? JSON.parse(monster.personalityParams)
20801   : monster.personalityParams;
20802
20803 return {
20804   id: monster.id,
20805   name: monster.name,
20806   style: monster.style,
20807   level: monster.level,
20808   exp: monster.exp,
20809   stamina: monster.stamina,
20810   maxStamina: monster.maxStamina,
20811   energy: monster.energy,
20812   maxEnergy: 1000000, // 能量无上限, 前端使用本地 maxPaiEnergy 逻辑
20813   personality: monster.personality,
20814   personalityParams,
20815   lastEnergyRecover: monster.lastEnergyRecover,
20816   lastStaminaRecover: monster.lastStaminaRecover
20817 };
20818 };
20819
20820 export const consumeStamina = async (userId: number, amount: number = 10): Promise<boolean> => {
20821   const success = await MonsterModel.consumeStamina(userId, amount);
20822   return success;
20823 };
20824
20825 export const recoverStamina = async (userId: number, amount: number = 20) => {
20826   await MonsterModel.recoverStamina(userId, amount);
20827   return { success: true };
20828 };
20829
20830 export const updateMonsterStamina = async (
20831   userId: number,
20832   delta: number
20833 ) => {
20834   console.log('[MonsterService] 更新体力 - 用户ID: ${userId}, 变化量: ${delta}');
20835   const monster = await MonsterModel.getMonsterByUserId(userId);
20836
20837   if (!monster) {
20838     console.log('[MonsterService] 更新体力失败 - 怪物不存在');
20839     return { success: false, message: '怪物不存在' };
20840   }
20841
20842   const newStamina = Math.max(0, Math.min(monster.maxStamina, monster.stamina + delta));
20843   await MonsterModel.updateMonster(monster.userId, { stamina: newStamina });
20844

```

```

20845 console.log('[MonsterService] 体力更新成功 - 旧值: ${monster.stamina}, 新值: ${newStamina}');
20846 return { success: true, stamina: newStamina };
20847 };
20848
20849 export const consumeEnergy = async (userId: number): Promise<boolean> => {
20850 console.log('[MonsterService] 消耗能量 - 用户ID: ${userId}');
20851 const success = await MonsterModel.consumeEnergy(userId);
20852 console.log('[MonsterService] 能量消耗${success ? '成功' : '失败'}');
20853 return success;
20854 };
20855
20856 export const consumeEnergyAmount = async (userId: number, amount: number): Promise<boolean> => {
20857 const success = await MonsterModel.consumeEnergyAmount(userId, amount);
20858 return success;
20859 };
20860
20861 export const recoverEnergy = async (userId: number, amount: number = 1) => {
20862 await MonsterModel.recoverEnergy(userId, amount);
20863 return { success: true };
20864 };
20865
20866 export const gainExp = async (userId: number, exp: number) => {
20867 console.log('[MonsterService] 获得经验 - 用户ID: ${userId}, 经验: ${exp}');
20868 const monster = await MonsterModel.getMonsterByUserId(userId);
20869
20870 if (!monster) {
20871 console.log('[MonsterService] 获得经验失败 - 怪物不存在');
20872 return { success: false, message: '怪物不存在' };
20873 }
20874
20875 const newExp = monster.exp + exp;
20876 const expToLevel = monster.level * 100;
20877 let newLevel = monster.level;
20878 let remainingExp = newExp;
20879
20880 while (remainingExp >= expToLevel) {
20881 remainingExp -= expToLevel;
20882 newLevel++;
20883 }
20884
20885 await MonsterModel.updateMonster(userId, {
20886 level: newLevel,
20887 exp: remainingExp,
20888 });
20889
20890 console.log('[MonsterService] 经验更新 - 等级: ${monster.level} -> ${newLevel}, 经验: ${monster.exp} -> ${remainingE
20891 return { success: true, level: newLevel, exp: remainingExp };
20892 };
20893
20894 export const chatWithMonster = async (userId: number, message: string) => {

```



```

20895 // 1. 保存用户消息
20896 await MonsterMessageModel.createMessage({
20897     userId,
20898     message,
20899     isUser: true
20900 });
20901
20902 // 2. 获取怪兽状态，确定性格类型
20903 const monsterStatus = await getMonsterStatus(userId);
20904 const personality: MonsterPersonalityType = monsterStatus?.personality || 'lively';
20905
20906 // 3. 构建对话消息（含 system prompt + 历史上下文）
20907 const messages = await buildMessages(userId, personality, message);
20908
20909 // 4. 调用 DeepSeek API
20910 let result: { content: string; tokens: number };
20911 try {
20912     result = await callDeepSeek(messages);
20913     console.log('[MonsterService] DeepSeek 回复 (tokens: ${result.tokens}): ${result.content.substring(0, 100)}.');
20914 } catch (error) {
20915     console.error('[MonsterService] DeepSeek 调用失败，使用预设回复:', error);
20916     // 降级：预设回复
20917     const fallbacks: Record<MonsterPersonalityType, string[]> = {
20918         lively: ['太棒了！继续加油哦~', '这个知识点很有趣呢！', '准备好开始冒险了吗？'],
20919         calm: ['慢慢来，学习需要耐心。', '这个知识点很重要，好好掌握。'],
20920         rebel: ['学习好无聊啊，我们去玩吧！', '这个也太难了吧...'],
20921     };
20922     const pool = fallbacks[personality];
20923     const content = pool[Math.floor(Math.random() * pool.length)];
20924     result = { content, tokens: Math.ceil(content.length / 2) };
20925 }
20926
20927 // 5. 扣除 能量：消耗 = 怪兽回复的字数 × 0.05
20928 const charCount = result.content.length;
20929 const energyCost = Math.max(1, Math.ceil(charCount * ENERGY_COST_PER_TOKEN));
20930 const currentEnergy = monsterStatus?.energy ?? 0;
20931 const newEnergy = Math.max(0, currentEnergy - energyCost);
20932 await MonsterModel.updateMonster(userId, { energy: newEnergy });
20933 console.log('[MonsterService] 能量扣除: ${energyCost} (字数: ${charCount}), 剩余: ${newEnergy}');
20934
20935 // 6. 保存怪兽回复
20936 await MonsterMessageModel.createMessage({
20937     userId,
20938     message: result.content,
20939     isUser: false
20940 });
20941
20942 return { message: result.content, tokens: result.tokens, energyCost, remainingEnergy: newEnergy };
20943 };
20944

```

```

20945 export const getMonsterMessages = async (userId: number) => {
20946   const messages = await MonsterMessageModel.getMessagesByUserId(userId);
20947   return { messages: messages.reverse() };
20948 };
20949
20950 === server/src/services/noteService.ts ===
20951
20952 export const createNote = async (userId: number, date: Date, content?: string) => {
20953   console.log('[NoteService] 创建笔记 - 用户ID: ${userId}, 日期: ${date}');
20954
20955   // 纯笔记存储，不再自动调用 AI 生成怪兽评论
20956   const notelId = await NoteModel.createNote({
20957     userId,
20958     date,
20959     content,
20960   });
20961
20962   console.log('[NoteService] 笔记创建成功 - 笔记ID: ${notelId}');
20963   return { success: true, notelId };
20964 };
20965
20966 export const getNotes = async (userId: number) => {
20967   console.log('[NoteService] 获取笔记列表 - 用户ID: ${userId}');
20968   const notes = await NoteModel.getNotesByUserId(userId);
20969   console.log('[NoteService] 获取到 ${notes.length} 条笔记');
20970   return { notes };
20971 };
20972
20973 export const getNoteByDate = async (userId: number, date: Date) => {
20974   console.log('[NoteService] 按日期获取笔记 - 用户ID: ${userId}, 日期: ${date}');
20975   const note = await NoteModel.getNoteByDate(userId, date);
20976   return note;
20977 };
20978
20979 export const updateNote = async (notelId: number, content?: string) => {
20980   console.log('[NoteService] 更新笔记 - 笔记ID: ${notelId}');
20981   await NoteModel.updateNote(notelId, { content });
20982   console.log('[NoteService] 笔记更新成功');
20983   return { success: true };
20984 };
20985
20986 === server/src/services/proService.ts ===
20987 import crypto from 'crypto';
20988 import { DatabaseConnection } from '../config/database';
20989
20989 const PLAN_DURATION: Record<string, number | null> = {
20990   monthly: 30,    // 30 days
20991   yearly: 365,    // 365 days
20992   lifetime: null, // permanent

```

```

20993 };
20994
20995 export class ProService {
20996   // 生成一批激活码
20997   static async generateCodes(count: number, planId: string, createdBy?: string): Promise<string[]> {
20998     if (!PLAN_DURATION[planId] && planId !== 'lifetime') {
20999       throw new Error('无效的套餐类型');
21000     }
21001
21002     const connection = await DatabaseConnection.getConnection();
21003     const codes: string[] = [];
21004
21005     for (let i = 0; i < count; i++) {
21006       const code = 'LF-' + crypto.randomBytes(6).toString('hex').toUpperCase();
21007       await connection.execute(
21008         'INSERT INTO activation_codes (code, plan_id, created_by, status) VALUES (?, ?, ?, ?)',
21009         [code, planId, createdBy || null, 'unused']
21010       );
21011       codes.push(code);
21012     }
21013
21014     console.log('[ProService] 生成 ${count} 个激活码, 套餐: ${planId}');
21015     return codes;
21016   }
21017
21018   // 验证并激活
21019   static async activateCode(code: string, userId: number): Promise<{
21020     success: boolean;
21021     planId?: string;
21022     expiresAt?: string;
21023   }> {
21024     const connection = await DatabaseConnection.getConnection();
21025
21026     // 查找未使用的激活码
21027     const [rows] = await connection.execute(
21028       'SELECT * FROM activation_codes WHERE code = ? AND status = ?',
21029       [code, 'unused']
21030     );
21031
21032     const record = (rows as any[][0]);
21033     if (!record) {
21034       throw new Error('激活码无效或已被使用');
21035     }
21036
21037     // 计算过期时间
21038     const duration = PLAN_DURATION[record.plan_id];
21039     let expiresAt: Date | null = null;
21040     if (duration !== null && duration !== undefined) {
21041       expiresAt = new Date(Date.now() + duration * 24 * 60 * 60 * 1000);
21042     }

```

```

21043
21044 // 更新激活码状态
21045 await connection.execute(
21046     'UPDATE activation_codes SET status = ?, used_by = ?, used_at = NOW() WHERE id = ?',
21047     ['used', userId, record.id]
21048 );
21049
21050 // 更新用户 Pro 状态
21051 if (expiresAt) {
21052     await connection.execute(
21053         'UPDATE users SET is_pro = 1, pro_activated_at = NOW(), pro_expires_at = ? WHERE id = ?',
21054         [expiresAt, userId]
21055     );
21056 } else {
21057     // 永久会员
21058     await connection.execute(
21059         'UPDATE users SET is_pro = 1, pro_activated_at = NOW(), pro_expires_at = NULL WHERE id = ?',
21060         [userId]
21061     );
21062 }
21063
21064 console.log('[ProService] 激活码使用成功 - 用户ID: ${userId}, 套餐: ${record.plan_id}, 过期: ${expiresAt || '永久'}');
21065 return {
21066     success: true,
21067     planId: record.plan_id,
21068     expiresAt: expiresAt?.toISOString(),
21069 };
21070 }
21071
21072 // 查询用户 Pro 状态
21073 static async getProStatus(userId: number): Promise<{
21074     isPro: boolean;
21075     planId?: string;
21076     activatedAt?: string;
21077     expiresAt?: string;
21078 }> {
21079     const connection = await DatabaseConnection.getConnection();
21080     const [rows] = await connection.execute(
21081         'SELECT is_pro, pro_activated_at, pro_expires_at FROM users WHERE id = ?',
21082         [userId]
21083     );
21084
21085     const user = (rows as any[][0]);
21086     if (!user) {
21087         throw new Error('用户不存在');
21088     }
21089
21090     // 检查是否过期
21091     if (user.is_pro && user.pro_expires_at && new Date(user.pro_expires_at) < new Date()) {
21092         // 已过期, 自动取消 Pro

```

```

21093     await connection.execute(
21094         'UPDATE users SET is_pro = 0 WHERE id = ?',
21095         [userId]
21096     );
21097     return { isPro: false };
21098 }
21099
21100 // 查询使用的激活码类型
21101 let planId: string | undefined;
21102 if (user.is_pro) {
21103     const [codeRows] = await connection.execute(
21104         'SELECT plan_id FROM activation_codes WHERE used_by = ? ORDER BY used_at DESC LIMIT 1',
21105         [userId]
21106     );
21107     const codeRecord = (codeRows as any[][0]);
21108     planId = codeRecord?.plan_id;
21109 }
21110
21111 return {
21112     isPro: !!user.is_pro,
21113     planId,
21114     activatedAt: user.pro_activated_at ? new Date(user.pro_activated_at).toISOString() : undefined,
21115     expiresAt: user.pro_expires_at ? new Date(user.pro_expires_at).toISOString() : undefined,
21116 };
21117 }
21118
21119 // 列出所有激活码（管理用）
21120 static async listCodes(filter?: { status?: string; planId?: string }): Promise<any[]> {
21121     const connection = await DatabaseConnection.getConnection();
21122     let sql = 'SELECT * FROM activation_codes WHERE 1=1';
21123     const params: any[] = [];
21124
21125     if (filter?.status) {
21126         sql += ' AND status = ?';
21127         params.push(filter.status);
21128     }
21129     if (filter?.planId) {
21130         sql += ' AND plan_id = ?';
21131         params.push(filter.planId);
21132     }
21133
21134     sql += ' ORDER BY created_at DESC LIMIT 100';
21135     const [rows] = await connection.execute(sql, params);
21136     return rows as any[];
21137 }
21138 }
21139
21140 === server/src/services/rewardService.ts ===
21140 import * as RewardModel from '../models/Reward';
21141

```

```

21142 export const createReward = async (userId: number, type: string, amount: number, source?: string) => {
21143   console.log('[RewardService] 创建奖励 - 用户ID: ${userId}, 类型: ${type}, 数量: ${amount}');
21144   const rewardId = await RewardModel.createReward({
21145     userId,
21146     type,
21147     amount,
21148     source
21149   });
21150   console.log('[RewardService] 奖励创建成功 - 奖励ID: ${rewardId}');
21151   return { success: true, rewardId };
21152 };
21153
21154 export const getRewards = async (userId: number) => {
21155   console.log('[RewardService] 获取奖励列表 - 用户ID: ${userId}');
21156   const rewards = await RewardModel.getRewardsByUserId(userId);
21157   console.log('[RewardService] 获取到 ${rewards.length} 条奖励');
21158   return { rewards };
21159 };
21160
21161 export const claimReward = async (rewardId: number) => {
21162   console.log('[RewardService] 领取奖励 - 奖励ID: ${rewardId}');
21163   const success = await RewardModel.claimReward(rewardId);
21164   console.log('[RewardService] 奖励领取${success ? '成功' : '失败'}');
21165   return { success };
21166 };
21167
=== server/src/services/skillService.ts ===
21168 import { SkillNode, SkillTreeListRequest, UserProgress, SkillTreeStats, SaveSkillTreeRequest } from '../types/
21169 import { DatabaseService } from '../databaseService';
21170 import { LLMService, LLMProvider } from '../llmService';
21171
21172 // 预置技能树数据（用于生成新技能树）
21173 const mockSkillTrees: Record<string, SkillNode> = {
21174   '前端开发': {
21175     id: '1',
21176     name: '前端开发',
21177     description: '前端开发技能树',
21178     children: [
21179       {
21180         id: '1-1',
21181         name: 'HTML/CSS',
21182         description: '网页基础结构',
21183         links: [
21184           { title: 'MDN HTML教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/HTML', type: 'documentatio
21185           { title: 'CSS教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/CSS', type: 'documentation' as
21186         ]
21187       },
21188       {
21189         id: '1-2',
21190         name: 'JavaScript',

```

```

21191     description: '前端编程语言',
21192     children: [
21193     {
21194         id: '1-2-1',
21195         name: '基础语法',
21196         description: '变量、函数、循环等',
21197         links: [
21198             { title: 'JavaScript教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/JavaScript', type: '
21199         ]
21200     },
21201     {
21202         id: '1-2-2',
21203         name: 'DOM操作',
21204         description: '文档对象模型操作',
21205         links: [
21206             { title: 'DOM教程', url: 'https://developer.mozilla.org/zh-CN/docs/Web/API/Document_Object_Model',
21207         ]
21208     }
21209 ]
21210 }
21211 ]
21212 },
21213 '后端开发': {
21214     id: '2',
21215     name: '后端开发',
21216     description: '后端开发技能树',
21217     children: [
21218     {
21219         id: '2-1',
21220         name: 'Node.js',
21221         description: 'JavaScript运行时',
21222         links: [
21223             { title: 'Node.js官方文档', url: 'https://nodejs.org/docs', type: 'documentation' as const }
21224         ]
21225     },
21226     {
21227         id: '2-2',
21228         name: '数据库',
21229         description: '数据存储和管理',
21230         children: [
21231             {
21232                 id: '2-2-1',
21233                 name: 'MySQL',
21234                 description: '关系型数据库',
21235                 links: [
21236                     { title: 'MySQL教程', url: 'https://dev.mysql.com/doc/', type: 'documentation' as const }
21237                 ]
21238             }
21239         ]
21240     }

```

```

21241   ]
21242   }
21243   };
21244
21245   // 初始化LLM服务 (根据环境变量配置)
21246   const initLLMService = (): LLMService | null => {
21247     const apiKey = process.env.LLM_API_KEY;
21248     const provider = process.env.LLM_PROVIDER as LLMProvider;
21249
21250     if (!apiKey || !provider) {
21251       console.warn('LLM service not configured. Using mock data instead.');
```

21252 return null;

21253 }

21254

21255 return new LLMService({

21256 provider,

21257 apiKey,

21258 baseURL: process.env.LLM_BASE_URL,

21259 model: process.env.LLM_MODEL

21260 });

21261 };

21262

21263 const llmService = initLLMService();

21264

21265 // 生成技能树 (优先使用LLM, 失败时回退到模拟数据)

21266 export const generateMockSkillTree = async (domain: string, level: string = 'beginner'): Promise<SkillNode> =>

21267 // 更新热门领域统计

21268 await DatabaseService.incrementDomainGeneratedCount(domain);

21269

21270 // 优先使用LLM生成技能树

21271 if (llmService) {

21272 try {

21273 console.log(`Generating skill tree for domain: \${domain}, level: \${level} using LLM`);

21274 const skillTree = await llmService.generateSkillTree({

21275 domain,

21276 level: level as 'beginner' | 'intermediate' | 'advanced',

21277 language: '中文',

21278 maxDepth: 4,

21279 includeResources: true

21280 });

21281

21282 console.log('Skill tree generated successfully by LLM');

21283 return skillTree;

21284 } catch (error) {

21285 console.warn('LLM generation failed, falling back to mock data:', error);

21286 console.error('LLM error details:', {

21287 message: error instanceof Error ? error.message : 'Unknown error',

21288 stack: error instanceof Error ? error.stack : 'No stack trace'

21289 });

21290 }


```

21291 }
21292
21293 // 回退到模拟数据
21294 console.log('Using mock data for domain: ${domain}');
21295 console.log('Available mock domains:', Object.keys(mockSkillTrees));
21296 let skillTree = mockSkillTrees[domain];
21297
21298 if (!skillTree) {
21299     // 如果领域不存在, 创建一个基础技能树
21300     skillTree = {
21301         id: generateId(),
21302         name: domain,
21303         description: `${domain}技能学习路径`,
21304         children: [
21305             {
21306                 id: generateId(),
21307                 name: '基础知识',
21308                 description: '学习该领域的基础概念',
21309                 links: [
21310                     { title: '入门指南', url: 'https://example.com', type: 'documentation' as const }
21311                 ]
21312             },
21313             {
21314                 id: generateId(),
21315                 name: '进阶学习',
21316                 description: '深入掌握核心技能',
21317                 links: [
21318                     { title: '进阶教程', url: 'https://example.com', type: 'documentation' as const }
21319                 ]
21320             }
21321         ]
21322     };
21323 }
21324
21325 return skillTree;
21326 };
21327
21328 // 获取技能树列表 (从数据库)
21329 export const getSkillTreeList = async (request: SkillTreeListRequest) => {
21330     const { page = 1, limit = 10, search, category } = request;
21331     console.log('[SkillService] 获取技能树列表 - 页码: ${page}, 搜索: ${search || '无'}, 分类: ${category || '无'}');
21332
21333     const result = await DatabaseService.getSkillTreesByUserId(1, page, limit);
21334
21335     return {
21336         trees: result.trees.map(tree => ({
21337             id: tree.id.toString(),
21338             name: tree.title,
21339             domain: tree.domain,
21340             description: tree.description,

```

```

21341     progress: tree.progress,
21342     createdAt: tree.createdAt
21343   })),
21344   total: result.total,
21345   page: result.page,
21346   limit: result.limit
21347 };
21348 };
21349
21350 // 根据ID获取技能树 (从数据库)
21351 export const getSkillTreeById = async (id: string) => {
21352   console.log('[SkillService] 获取技能树详情 - ID: ${id}');
21353   const tree = await DatabaseService.getSkillTreeById(parseInt(id));
21354
21355   if (!tree) {
21356     console.log('[SkillService] 技能树不存在 - ID: ${id}');
21357     return null;
21358   }
21359
21360   return {
21361     id: tree.id.toString(),
21362     name: tree.title,
21363     domain: tree.domain,
21364     description: tree.description,
21365     nodes: tree.nodes,
21366     progress: tree.progress,
21367     createdAt: tree.createdAt
21368   };
21369 };
21370
21371 // 保存用户技能树 (到数据库)
21372 export const saveUserSkillTree = async (request: SaveSkillTreeRequest) => {
21373   const { userId, skillTree, title, tags } = request;
21374   console.log('[SkillService] 保存技能树 - 用户ID: ${userId}, 标题: ${title || skillTree.name}');
21375
21376   try {
21377     const treeld = await DatabaseService.createSkillTree({
21378       userId: parseInt(userId),
21379       domain: skillTree.name,
21380       title: title || skillTree.name,
21381       description: skillTree.description,
21382       nodes: [skillTree],
21383       isPublic: false
21384     });
21385
21386     console.log('[SkillService] 技能树保存成功 - 树ID: ${treeld}');
21387     return { success: true, treeld: treeld.toString() };
21388   } catch (error) {
21389     console.error('[SkillService] 保存技能树失败:', error);
21390     return { success: false, error: '保存失败' };

```

```

21391 }
21392 };
21393
21394 // 获取用户进度 (从数据库)
21395 export const getUserProgress = async (userId: string, skillTreeld?: string): Promise<UserProgress[]> => {
21396   if (skillTreeld) {
21397     const progress = await DatabaseService.getUserProgress(parseInt(userId), parseInt(skillTreeld));
21398     return [{
21399       userId,
21400       skillTreeld,
21401       completedNodes: [], // 需要从学习记录计算
21402       completedLinks: [], // 需要从学习记录计算
21403       progress: progress.overallProgress,
21404       lastUpdated: progress.lastUpdated
21405     }];
21406   }
21407
21408   // 获取用户所有技能树的进度
21409   const trees = await DatabaseService.getSkillTreesByUserId(parseInt(userId));
21410   const progresses: UserProgress[] = [];
21411
21412   for (const tree of trees.trees) {
21413     const progress = await DatabaseService.getUserProgress(parseInt(userId), tree.id);
21414     progresses.push({
21415       userId,
21416       skillTreeld: tree.id.toString(),
21417       completedNodes: [],
21418       completedLinks: [],
21419       progress: progress.overallProgress,
21420       lastUpdated: progress.lastUpdated
21421     });
21422   }
21423
21424   return progresses;
21425 };
21426
21427 // 更新用户进度 (到数据库)
21428 export const updateUserProgress = async (userId: string, skillTreeld: string, completedNodes: string[], comple
21429   // 这里简化处理, 实际应该更新学习记录表
21430   const progress = Math.round((completedNodes.length / 10) * 100); // 简化计算
21431
21432   await DatabaseService.updateSkillTreeProgress(parseInt(skillTreeld), progress);
21433
21434   return {
21435     userId,
21436     skillTreeld,
21437     completedNodes,
21438     completedLinks,
21439     progress,
21440     lastUpdated: new Date()

```

```

21441   };
21442 };
21443
21444 // 获取统计信息（从数据库）
21445 export const getStatistics = async (): Promise<SkillTreeStats> => {
21446   console.log('[SkillService] 获取统计信息');
21447   const popularDomains = await DatabaseService.getPopularDomains(5);
21448
21449   return {
21450     totalTrees: 100,
21451     popularDomains: popularDomains.map(d => ({ domain: d.domain, count: d.popularity })),
21452     averageProgress: 45,
21453     totalUsers: 50
21454   };
21455 };
21456
21457 export const searchPopularDomains = async (keyword: string) => {
21458   console.log('[SkillService] 搜索热门领域 - 关键词: ${keyword}');
21459   await DatabaseService.incrementDomainSearchCount(keyword);
21460
21461   const domains = await DatabaseService.getPopularDomains(10);
21462   const results = domains
21463     .filter(d => d.domain.toLowerCase().includes(keyword.toLowerCase()))
21464     .map(d => d.domain);
21465
21466   console.log('[SkillService] 搜索结果: ${results.length} 条');
21467   return {
21468     keyword,
21469     results,
21470     total: results.length
21471   };
21472 };
21473
21474 export const getRecommendedPath = async (domain: string, currentLevel: string, targetLevel: string) => {
21475   console.log('[SkillService] 获取推荐路径 - 领域: ${domain}, 当前等级: ${currentLevel}, 目标等级: ${targetLevel}');
21476   return {
21477     domain,
21478     currentLevel,
21479     targetLevel,
21480     path: [
21481       { step: 1, topic: '基础知识', estimatedTime: '2周' },
21482       { step: 2, topic: '核心概念', estimatedTime: '3周' },
21483       { step: 3, topic: '实战项目', estimatedTime: '4周' }
21484     ],
21485     totalEstimatedTime: '9周'
21486   };
21487 };
21488
21489 export const getUserLearningReport = async (userId: string, period: string = 'month') => {
21490   console.log('[SkillService] 获取学习报告 - 用户ID: ${userId}, 周期: ${period}');

```

```

21491     return {
21492         userId,
21493         period,
21494         summary: {
21495             totalLearningTime: 120,
21496             completedSkills: 15,
21497             averageProgress: 65,
21498             streakDays: 7
21499         },
21500         recommendations: [
21501             '建议每天保持30分钟学习时间',
21502             '重点关注前端开发的核心概念',
21503             '尝试完成一个小型实战项目'
21504         ]
21505     };
21506 };
21507
21508 // 辅助函数: 生成唯一ID
21509 const generateId = (): string => {
21510     return Date.now().toString(36) + Math.random().toString(36).substr(2);
21511 };
21512
21513 === server/src/types/index.ts ===
21514 export * from './skill';
21515 === server/src/types/skill.ts ===
21516 export interface MonsterPersonality {
21517     cheerful: number;
21518     calm: number;
21519     rebellious: number;
21520 }
21521
21522 export interface SkillNode {
21523     id: string;
21524     name: string;
21525     description: string;
21526     children?: SkillNode[];
21527     links?: SkillLink[];
21528 }
21529
21530 export interface SkillLink {
21531     title: string;
21532     url: string;
21533     type: 'video' | 'article' | 'course' | 'documentation';
21534 }
21535
21536 export interface SkillTreeRequest {
21537     domain: string;
21538     level?: 'beginner' | 'intermediate' | 'advanced';
21539 }
21540
21541 export interface UserProgress {

```

```

21539   userId: string;
21540   skillTreeId: string;
21541   completedNodes: string[];
21542   completedLinks: string[];
21543   progress: number;
21544   lastUpdated: Date;
21545 }
21546
21547 export interface SkillTreeListRequest {
21548   page?: number;
21549   limit?: number;
21550   search?: string;
21551   category?: string;
21552 }
21553
21554 export interface SkillTreeStats {
21555   totalTrees: number;
21556   popularDomains: Array<{domain: string; count: number}>;
21557   averageProgress: number;
21558   totalUsers: number;
21559 }
21560
21561 export interface SaveSkillTreeRequest {
21562   userId: string;
21563   skillTree: SkillNode;
21564   title?: string;
21565   tags?: string[];
21566 }
21567
21568 export interface LearningReport {
21569   userId: string;
21570   period: string;
21571   summary: {
21572     totalLearningTime: string;
21573     completedSkills: number;
21574     averageProgress: number;
21575     streakDays: number;
21576   };
21577   weeklyProgress: Array<{week: string; progress: number}>;
21578   topSkills: string[];
21579   recommendations: string[];
21580 }
21581
21582 export interface RecommendedPath {
21583   domain: string;
21584   currentLevel: string;
21585   targetLevel: string;
21586   path: Array<{step: number; topic: string; estimatedTime: string}>;
21587   totalEstimatedTime: string;
21588 }

```